

Projet M1 — Structures de données avancées

Route planning : répondre vite à des requêtes de plus court chemin

Groupe de 3 — Durée : 6 semaines

1 Idée générale

Dans beaucoup d'applications (GPS, logistique, jeux, cartographie), on a toujours le même besoin : *on veut calculer rapidement un itinéraire entre deux points*. Le problème, c'est que sur un réseau routier réaliste, un algorithme correct mais "simple" comme Dijkstra devient trop lent dès qu'on enchaîne un grand nombre de requêtes.

Le but du projet est donc de construire un petit moteur de *route planning* : on charge un graphe routier, on pose des requêtes ($s \rightarrow t$), et on compare plusieurs méthodes qui accélèrent les réponses grâce à des **structures de données** et du **prétraitement**.

2 Ce qu'on veut montrer

On ne cherche pas seulement à "faire marcher" un algo. On veut être capables de répondre à des questions du type :

- Qu'est-ce qui fait gagner du temps : l'heuristique ? la représentation mémoire ? le prétraitement ?
- Combien coûte l'accélération (en mémoire et en temps de prétraitement) ?
- Est-ce que le gain est stable selon les types de requêtes (courtes vs longues distances) ?

3 Données et modélisation

3.1 Graphe

On modélise le réseau comme un graphe orienté pondéré $G = (V, E)$:

- un noeud = une intersection,
- une arête = un segment de route,
- un poids = distance (m) ou temps estimé (s).

3.2 Jeu de données

Deux possibilités :

- **Données réelles** : extraction OpenStreetMap sur une zone (ville ou région), puis conversion en graphe.
- **Données synthétiques** : graphe type grille perturbée (utile pour tester vite et contrôler la taille).

Dans l'idéal, on vise un graphe suffisamment grand pour que les optimisations aient un vrai impact (au moins plusieurs centaines de milliers d'arêtes).

4 Approche technique : algorithmes + structures de données

4.1 Représentation du graphe

C'est un point central du projet : sur des gros graphes, la structure mémoire compte énormément. On utilisera une représentation compacte (par exemple de type CSR : tableaux contigus + offsets), afin de :

- limiter la mémoire,
- améliorer la locality cache,
- accélérer l'accès aux voisins.

4.2 Baseline : Dijkstra (référence)

Dijkstra sert de point de comparaison : c'est notre version "sûre". Le coeur est une file de priorité (tas binaire), et on instrumente l'exécution (nombre d'extractions, relaxations, temps total).

4.3 A* (heuristique admissible)

A* modifie Dijkstra en ajoutant une heuristique $h(v)$ qui estime le coût restant. Sur un graphe routier, une heuristique naturelle est la distance "à vol d'oiseau" (euclidienne/haversine), éventuellement divisée par une vitesse maximale pour rester admissible. L'attendu : moins de noeuds explorés, donc des requêtes plus rapides.

4.4 ALT : A* avec landmarks

ALT améliore A* en utilisant des *landmarks* (points repères). Après un prétraitement (distances vers/depuis les landmarks), on obtient une heuristique plus informative :

$$h(v, t) = \max_{\ell \in L} |d(\ell, t) - d(\ell, v)|.$$

On étudiera l'effet du nombre de landmarks et de leur sélection. L'attendu : encore moins d'exploration qu'avec A* simple, mais au prix d'un stockage supplémentaire.

4.5 Contraction Hierarchies (CH)

CH est une méthode plus "ingénierie" : on investit du temps en prétraitement pour rendre les requêtes très rapides. L'idée est de contracter les noeuds dans un certain ordre et d'ajouter des *raccourcis* pour préserver les plus courts chemins. Ensuite, une requête se fait via une recherche bidirectionnelle, restreinte aux arêtes "montantes". C'est typiquement la méthode qui brille sur des lots de requêtes.

5 Évaluation : comment on compare proprement

5.1 Mesures

Pour chaque méthode (Dijkstra, A*, ALT, CH), on mesurera :

- temps par requête (moyenne, p50, p95),
- débit (requêtes/s sur un lot),
- mémoire (taille du graphe + structures additionnelles),
- coût de prétraitement (ALT/CH).

5.2 Protocole

- Générer un lot de paires (s, t) reproductible (graine fixe).
- Mélanger des requêtes courtes/moyennes/longues distances.
- Vérifier la correction : même distance retournée que Dijkstra (à tolérance près si poids flottants).
- Pour les analyses, regarder les plans de recherche : noeuds explorés, relaxations, etc.

6 Organisation (6 semaines, 3 personnes)

6.1 Répartition

- **A (données & graphe)** : extraction/génération, parsing, représentation CSR/compacte, validation.
- **B (algos de base)** : Dijkstra + A* + ALT, tests de correction, choix heuristiques/landmarks.
- **C (CH & mesures)** : CH (prétraitement + requête), framework benchmark, tableaux/figures.

6.2 Planning

- S1** Mise en place : chargement du graphe, représentation, Dijkstra + tests.
- S2** A* + premiers benchmarks, vérification sur cas simples.
- S3** ALT : sélection des landmarks, prétraitement, comparaison.
- S4** CH : contraction + raccourcis, requêtes CH, validation.
- S5** Optimisation & profiling : mémoire, structures, micro-gains mesurés.
- S6** Consolidation : rapport, figures, tableau comparatif final, démo.

7 Livrables

- Code + README + scripts (exécution et benchmark reproductibles).
- Données (ou script d'extraction) + lot de requêtes.
- Résultats (CSV) + figures + tableau de synthèse.
- Rapport (8–12 pages) : démarche, choix, mesures, discussion des compromis.

8 Extensions (si on a le temps)

- Comparer plusieurs structures de file de priorité (tas binaire vs variantes).
- Tester plusieurs ordres de contraction (CH) ou stratégies de landmarks (ALT).
- Version “temps de trajet” avec vitesses par type de route.